# LAPORAN PRAKTIKUM IOT TRANSPORT LAYER

*Transport Layer: TCP, UDP, Message Framing, dan Reliabilitas Aplikasi.*

https://github.com/ridlofw/transport-layer


| | |
|---|---|
| Nama Mahasiswa | : Ridlo Fanata Wicaksana |
| NIM | : 26/591572/NPA/20034 |
| Nama Mahasiswa | : Javier Gavra Abhinaya |
| NIM | : 26/592020/NPA/20050 |
| Nama Mahasiswa | : Nur Alif Maulana Syafrudin |
| NIM | : 26/591608/NPA/20042 |
| Mata Kuliah | : Internet of Things |


## RINGKASAN PROYEK

Laporan ini menyajikan analisis menyeluruh dari eksekusi lima eksperimen utama praktikum Transport Layer dalam konteks sistem Internet of Things (IoT). Praktikum ini bertujuan untuk mengevaluasi secara langsung perbedaan mendasar antara protokol Transmission Control Protocol (TCP) yang berbasis byte stream terurut dan User Datagram Protocol (UDP) yang berbasis datagram independen. Selain itu, praktikum mendemonstrasikan perancangan mekanisme reliabilitas tingkat aplikasi (Application-Level Acknowledgment dan Retry), teknik message framing, serta inspeksi header paket menggunakan tcpdump pada antarmuka jaringan virtual.

- Tujuan 1: Mengidentifikasi transport ports dan sockets pada sistem operasi Linux.
- Tujuan 2: Membedakan model data TCP byte stream dengan UDP datagram independen.
- Tujuan 3: Mengimplementasikan mekanisme Acknowledgment (ACK) dan Bounded Retry di tingkat aplikasi.
- Tujuan 4: Menginterpretasikan metrik pengiriman, timeout, deduplikasi, dan latensi tanpa mengacaukan perilaku transport dengan aplikasi.

## BAB 1: PENDAHULUAN DAN PERSIAPAN LINGKUNGAN KERJA

### 1.1 Latar Belakang dan Capaian Pembelajaran

Dalam arsitektur jaringan IoT, pemrosesan telemetry dan pengiriman command antar smart objects sangat bergantung pada protokol di lapisan transport. TCP menyediakan garansi transmisi data yang andal dan berurutan melalui mekanisme retransmisi internal dan flow control. Namun, garansi TCP berakhir pada buffer socket penerima, TCP tidak memberikan konfirmasi apakah aplikasi penerima berhasil memproses pesan bisnis tersebut. Sebaliknya, UDP bersifat connectionless dan tidak memiliki garansi retransmisi internal, namun memberikan kebebasan bagi aplikasi IoT untuk menerapkan strategi reliabilitas yang disesuaikan dengan toleransi latensi dan efisiensi daya baterai sensor node.

### 1.2 Konsep Dasar Socket, Port, dan Loopback

Sebuah socket merupakan antarmuka sistem operasi yang digunakan oleh aplikasi untuk mengirim dan menerima data melalui jaringan. Alamat IP mengidentifikasi antarmuka jaringan, sedangkan nomor port mengidentifikasi endpoint transport spesifik. Pada praktikum ini, sensor virtual mengirimkan rekaman JSON ke server pengumpul pada port 5005. Alamat IP 127.0.0.1 merupakan antarmuka loopback lokal, di mana proses klien dan server berjalan pada mesin virtual yang sama.

### 1.3 Persiapan Lingkungan Virtual dan Penginstalan Paket

Lingkungan kerja yang digunakan adalah Ubuntu 24.04 LTS di atas Windows Subsystem for Linux (WSL2). Eksekusi dilakukan di dalam direktori kerja terisolasi ~/iot-lab. Paket-paket pendukung utama diinstal menggunakan manajer paket apt:

```
mkdir -p ~/iot-lab && cd ~/iot-lab
sudo apt update
sudo apt install -y python3 iproute2 tcpdump
python3 --version
```

Verifikasi menunjukkan Python versi 3.14.4 beserta utilitas iproute2 dan tcpdump telah terpasang dengan sempurna di dalam WSL Ubuntu.

### 1.4 Berkas Program Utama

Praktikum mengandalkan dua berkas Python utama yang memanfaatkan pustaka standar Python 3 (standard library only):

1. framing_demo.py: Program demonstrasi untuk membuktikan bahwa dua pemanggilan sendall() pada TCP tidak menghasilkan dua rekaman penerimaan utuh pada recv().
2. transport_lab.py: Program pengujian telemetri IoT (klien dan server) yang mendukung protokol TCP dan UDP, dilengkapi fitur fault injection (simulasi data drop dan ACK drop), pencatatan log CSV per pengiriman, serta mekanisme stop-and-wait ACK/retry.

## BAB 2: EKSPERIMEN 1 - MESSAGE FRAMING TCP

### 2.1 Tujuan Eksperimen

Eksperimen 1 bertujuan membuktikan bahwa pemanggilan recv() pada socket TCP tidak menjamin pembatasan pesan (message boundary). TCP menangani data sebagai aliran byte (byte stream) berurutan tanpa mengetahui struktur rekaman aplikasi.

### 2.2 Pelaksanaan Eksperimen dengan Buffer 4 Byte

Server dijalankan pada Terminal A dengan pemanggilan conn.recv(4) secara berulang. Klien pada Terminal B mengirim dua pesan terpisah melalui dua pemanggilan sendall():

```
# Terminal A
python3 framing_demo.py server
```

```
# Terminal B
python3 framing_demo.py client
```

Data sebesar 21 byte (b'temperature=25\n' dan b'humidity=60\n') diterima oleh server dalam 7 potongan byte terfragmentasi (b'temp', b'erat', b'ure=', b'25\nh', b'umid', b'ity=', b'60\n'). Aplikasi server merekonstruksi kembali dua rekaman utuh tersebut hanya karena adanya karakter pemisah newline (\n) yang dipisah menggunakan splitlines().

```
ridlo@lenovo:~/iot-lab$ python3 framing_demo.py server
READY on port 5006
recv: b'temp'
recv: b'erat'
recv: b'ure='
recv: b'25\nh'
recv: b'umid'
recv: b'ity='
recv: b'60\n'
records: [b'temperature=25', b'humidity=60']
ridlo@lenovo:~/iot-lab$
```

### 2.3 Checkpoint: Pengujian dengan Ukuran Buffer 3 Byte

Sesuai instruksi checkpoint pada tutorial, ukuran buffer diubah menjadi conn.recv(3) menggunakan perintah sed:

```
sed -i 's/conn.recv(4)/conn.recv(3)/' framing_demo.py
python3 framing_demo.py server
```

Dengan buffer 3 byte, data 21 byte diterima dalam 9 potongan terfragmentasi (b'tem', b'per', b'atu', b're=', b'25\n', b'hum', b'idi', b'ty=', b'60\n'). Meskipun jumlah potongan bertambah dari 7 menjadi 9, hasil akhir pembacaan rekaman records: [b'temperature=25', b'humidity=60'] tetap identik.

```
ridlo@lenovo::~/iot-lab$ cd ~/iot-lab
sed -i 's/conn.recv(4)/conn.recv(3)/' framing_demo.py
grep recv framing_demo.py
                part = conn.recv(3)
                print("recv:", repr(part))
ridlo@lenovo:~/iot-lab$ python3 framing_demo.py server
READY on port 5006
recv: b'tem'
recv: b'per'
recv: b'atu'
recv: b're='
recv: b'25\n'
recv: b'hum'
recv: b'idi'
recv: b'ty='
recv: b'60\n'
records: [b'temperature=25', b'humidity=60']
ridlo@lenovo:~/iot-lab$
```

### 2.4 Analisis Kritis Byte Stream dan Framing Biner

Hasil eksperimen 1 menegaskan bahwa pemanggilan recv(4096) atau ukuran buffer berapapun pada TCP tidak pernah bisa diasumsikan secara aman sebagai 'satu pembacaan sensor lengkap'. Potongan byte yang diterima bergantung pada segmentasi TCP, MTU jaringan, buffer OS, dan timing kedatangan paket. Untuk data tekstual, penggunaan karakter pemisah (delimiter) seperti newline bersifat efektif. Namun, untuk data biner IoT tanpa pemisah karakter yang aman, pendekatan yang disyaratkan adalah Length-Prefix Framing: pengirim menyisipkan header panjang data (misalnya 2 atau 4 byte integer) di depan setiap payload, sehingga penerima tahu pasti berapa byte yang harus dibaca sebelum memproses satu rekaman utuh.

## BAB 3: EKSPERIMEN 2 - BASELINE TELEMETRI TCP DAN UDP

### 3.1 Tujuan dan Skenario Pengujian Baseline

Eksperimen 2 mengukur performa pengiriman telemetri dasar (baseline) sebanyak 20 pembacaan suhu menggunakan pola stop-and-wait workload (klien mengirim satu data, menunggu ACK aplikasi dari server, lalu melanjutkan ke data berikutnya) pada protokol UDP dan TCP tanpa adanya gangguan jaringan.

### 3.2 Pengujian Baseline UDP dan TCP

Perintah eksekusi untuk kedua protokol:

```
# Pengujian UDP Baseline
python3 transport_lab.py server --protocol udp
python3 transport_lab.py client --protocol udp --out udp_baseline.csv

# Pengujian TCP Baseline
```

```
python3 transport_lab.py server --protocol tcp
python3 transport_lab.py client --protocol tcp --timeout 5 --out tcp_baseline.csv
```

Pada kondisi jaringan lokal yang sehat, kedua protokol mencapai tingkat keberhasilan sempurna yaitu acknowledged=20/20 dengan application_sends=20 tanpa adanya retransmisi ulang.

```
ridlo@lenovo:~/iot-lab$ explorer.exe .
ridlo@lenovo:~/iot-lab$ python3 transport_lab.py client --protocol udp --out udp_baseline.csv
run=C79a141b acknowledged=20/20
application_sends= 20
CSV: udp_baseline.csv
ridlo@lenovo:~/iot-lab$ python3 transport_lab.py client --protocol tcp --timeout 5 --out tcp_baseline.csv
run=5b4991f7 acknowledged=20/20
application_sends= 20
CSV: tcp_baseline.csv
```

```
ridlo@lenovo:/mnt/c/WINDOWS/system32$ cd ~/iot-lab
ridlo@lenovo:~/iot-lab$ python3 transport_lab.py server --protocol udp
READY udp 127.0.0.1 5005
STORE ('c79a141b', 1) total_unique= 1
STORE ('c79a141b', 2) total_unique= 2
STORE ('c79a141b', 3) total_unique= 3
STORE ('c79a141b', 4) total_unique= 4
STORE ('c79a141b', 5) total_unique= 5
STORE ('c79a141b', 6) total_unique= 6
STORE ('c79a141b', 7) total_unique= 7
STORE ('c79a141b', 8) total_unique= 8
STORE ('c79a141b', 9) total_unique= 9
STORE ('c79a141b', 10) total_unique= 10
STORE ('c79a141b', 11) total_unique= 11
STORE ('c79a141b', 12) total_unique= 12
STORE ('c79a141b', 13) total_unique= 13
STORE ('c79a141b', 14) total_unique= 14
STORE ('c79a141b', 15) total_unique= 15
STORE ('c79a141b', 16) total_unique= 16
STORE ('c79a141b', 17) total_unique= 17
STORE ('c79a141b', 18) total_unique= 18
STORE ('c79a141b', 19) total_unique= 19
STORE ('c79a141b', 20) total_unique= 20
```

```
ridlo@lenovo:~/iot-lab$ python3 transport_lab.py server --protocol tcp
READY tcp 127.0.0.1 5005
STORE ('5b4991f7', 1) total_unique= 1
STORE ('5b4991f7', 2) total_unique= 2
STORE ('5b4991f7', 3) total_unique= 3
STORE ('5b4991f7', 4) total_unique= 4
STORE ('5b4991f7', 5) total_unique= 5
STORE ('5b4991f7', 6) total_unique= 6
STORE ('5b4991f7', 7) total_unique= 7
STORE ('5b4991f7', 8) total_unique= 8
STORE ('5b4991f7', 9) total_unique= 9
STORE ('5b4991f7', 10) total_unique= 10
STORE ('5b4991f7', 11) total_unique= 11
STORE ('5b4991f7', 12) total_unique= 12
STORE ('5b4991f7', 13) total_unique= 13
STORE ('5b4991f7', 14) total_unique= 14
STORE ('5b4991f7', 15) total_unique= 15
STORE ('5b4991f7', 16) total_unique= 16
STORE ('5b4991f7', 17) total_unique= 17
STORE ('5b4991f7', 18) total_unique= 18
STORE ('5b4991f7', 19) total_unique= 19
STORE ('5b4991f7', 20) total_unique= 20
```

### 3.3 Analisis Metrik elapsed_ms dan Stop-and-Wait Workload

Server menggunakan identifier unik berupa UUID acak (run) untuk membedakan klien yang berbeda. Nilai elapsed_ms yang tercatat pada file CSV mencakup seluruh waktu penyelesaian RTT aplikasi (mulai dari data dikirim, diproses server, hingga ACK diterima kembali oleh klien). Ini merupakan Waktu Penyelesaian Aplikasi (Application Round-Trip Completion Time), dan bukan sekadar latensi jaringan satu arah.

## BAB 4: EKSPERIMEN 3 - DATA HILANG DAN RETRY TERBATAS

### 4.1 Simulasi Fault Injection Data Drop

Eksperimen 3 menguji ketahanan aplikasi terhadap kehilangan data di jaringan. Server dikonfigurasi dengan fault injector --drop-every 4 --fault data yang secara sengaja mengabaikan kedatangan pertama dari sequence number kelipatan 4 (seq 4, 8, 12, 16, dan 20).

### 4.2 Pengujian Tanpa Retry vs Dengan Retry 3 Kali

Eksekusi dilakukan dalam dua tahap pada server yang sama:

```
# Server dengan Data Drop
python3 transport_lab.py server --protocol udp --drop-every 4 --fault data

# Klien Tanpa Retry (--retries 0)
python3 transport_lab.py client --protocol udp --retries 0 --out udp_drop.csv

# Klien Dengan Retry 3 Kali (--retries 3)
python3 transport_lab.py client --protocol udp --retries 3 --out udp_retry.csv
```

Pada pengujian tanpa retry (--retries 0), klien mengalami timeout pada 5 paket yang dijatuhkan sehingga hanya mencapai acknowledged=15/20. Ketika retry diaktifkan (--retries 3), klien berhasil memulihkan 5 paket yang hilang melalui percobaan ulang, mencapai acknowledged=20/20 dengan total pengiriman application_sends=25 (20 pengiriman utama + 5 retransmisi).

```
ridlo@lenovo:~/iot-lab$ python3 transport_lab.py server --protocol udp --drop-every 4 --fault data
 READY udp 127.0.0.1 5005
 STORE ('17015cb6', 1) total_unique= 1
 STORE ('17015cb6', 2) total_unique= 2
 STORE ('17015cb6', 3) total_unique= 3
 SIMULATED_DATA_DROP ('17015cb6', 4)
 STORE ('17015cb6', 5) total_unique= 4
 STORE ('17015cb6', 6) total_unique= 5
 STORE ('17015cb6', 7) total_unique= 6
 SIMULATED_DATA_DROP ('17015cb6', 8)
 STORE ('17015cb6', 9) total_unique= 7
 STORE ('17015cb6', 10) total_unique= 8
 STORE ('17015cb6', 11) total_unique= 9
 SIMULATED_DATA_DROP ('17015cb6', 12)
 STORE ('17015cb6', 13) total_unique= 10
 STORE ('17015cb6', 14) total_unique= 11
 STORE ('17015cb6', 15) total_unique= 12
 SIMULATED_DATA_DROP ('17015cb6', 16)
 STORE ('17015cb6', 17) total_unique= 13
 STORE ('17015cb6', 18) total_unique= 14
 STORE ('17015cb6', 19) total_unique= 15
 SIMULATED_DATA_DROP ('17015cb6', 20)
 STORE ('05ef8785', 1) total_unique= 16
 STORE ('05ef8785', 2) total_unique= 17
 STORE ('05ef8785', 3) total_unique= 18
 SIMULATED_DATA_DROP ('05ef8785', 4)
 STORE ('05ef8785', 4) total_unique= 19
 STORE ('05ef8785', 5) total_unique= 20
 STORE ('05ef8785', 6) total_unique= 21
 STORE ('05ef8785', 7) total_unique= 22
```

```
ridlo@lenovo:~/iot-lab$ python3 transport_lab.py client --protocol udp --retries 0 --out udp_drop.csv
run=17015cb6 acknowledged=15/20
application_sends= 20
CSV: udp_drop.csv
ridlo@lenovo:~/iot-lab$ python3 transport_lab.py client --protocol udp --retries 3 --out udp_retry.csv
run=05ef8785 acknowledged=20/20
application_sends= 25
CSV: udp_retry.csv
ridlo@lenovo:~/iot-lab$
```

## 4.3 Evaluasi Variasi Durasi Timeout

Pengujian retry diulang menggunakan variasi timeout 0.10s, 0.25s (default), dan 0.50s untuk menganalisis dampaknya terhadap latensi aplikasi:

| Nilai Timeout (detik) | Median Completion Time (ms) | Maksimum Completion Time (ms) |
|---|---|---|

| | | |
|---|---|---|
| 0.10 s | 0.602 ms | 101.364 ms |
| 0.25 s (Default) | 0.604 ms | 251.996 ms |
| 0.50 s | 0.578 ms | 502.486 ms |

Analisis menunjukkan bahwa nilai Median Completion Time tetap stabil di kisaran ~0.6 ms karena 15 dari 20 paket langsung berhasil pada percobaan pertama. Namun, nilai Maksimum Completion Time naik secara proporsional dengan durasi timeout, karena 5 paket yang dijatuhkan harus menunggu durasi timeout penuh sebelum retransmisi dikirimkan.

### 4.4 Bahaya Timeout Terlalu Pendek dan Unbounded Retries

Timeout yang terlalu singkat berisiko menganggap paket yang sedang dalam perjalanan (delayed) sebagai hilang. Hal ini memicu retransmisi palsu (spurious retries) yang memboroskan bandwidth dan energi baterai sensor IoT. Di sisi lain, mencoba tanpa batas (unbounded retries / retry forever) sangat berbahaya pada jaringan IoT: jika terjadi kegagalan permanen (server mati atau putus koneksi), sensor akan terus mencoba tanpa henti hingga memori dan baterai habis sepenuhnya. Bounded retries (--retries N) memberikan batas usaha yang rasional sebelum melaporkan kegagalan ke lapisan atas.

## BAB 5: EKSPERIMEN 4 - KEHILANGAN ACK DAN DEDUPLIKASI SERVER

### 5.1 Simulasi Fault Injection ACK Drop

Eksperimen 4 menguji kondisi di mana data sensor berhasil diterima dan disimpan oleh server, namun balasan konfirmasi (ACK) hilang di perjalanan menuju klien. Server dikonfigurasi dengan --fault ack --drop-every 4.

### 5.2 Pengujian Klien Tanpa Retry vs Dengan Retry 3 Kali

Hasil eksekusi tanpa retry (--retries 0) menunjukkan bahwa server telah menyimpan seluruh 20 data unik (total_unique=20). Namun, karena 5 ACK dijatuhkan (SIMULATED_ACK_DROP), klien mengalami timeout dan menganggap pengiriman gagal (acknowledged=15/20). Ini membuktikan bahwa hilangnya ACK tidak membatalkan penyimpanan data yang sudah terjadi di server.

```
# Server ACK Drop
python3 transport_lab.py server --protocol udp --drop-every 4 --fault ack

# Klien Retry 3 Kali
python3 transport_lab.py client --protocol udp --retries 3 --out ack_retry.csv
```

Ketika retry diaktifkan (--retries 3), klien mengirim ulang 5 paket yang tidak ber-ACK. Pada server, muncul log DUPLICATE untuk paket kiriman ulang tersebut. Server mengenali pasangan (run, seq) yang sudah

ada di memori, menolak menyimpan data duplikat (total_unique tidak bertambah), namun tetap membalas ACK agar klien mengetahui keberhasilan pengiriman.

```
SIMULATED_ACK_DROP ('a4a42903', 4)
DUPLICATE ('a4a42903', 4) total_unique= 24
STORE ('a4a42903', 5) total_unique= 25
STORE ('a4a42903', 6) total_unique= 26
STORE ('a4a42903', 7) total_unique= 27
STORE ('a4a42903', 8) total_unique= 28
SIMULATED_ACK_DROP ('a4a42903', 8)
DUPLICATE ('a4a42903', 8) total_unique= 28
STORE ('a4a42903', 9) total_unique= 29
STORE ('a4a42903', 10) total_unique= 30
STORE ('a4a42903', 11) total_unique= 31
STORE ('a4a42903', 12) total_unique= 32
SIMULATED_ACK_DROP ('a4a42903', 12)
DUPLICATE ('a4a42903', 12) total_unique= 32
STORE ('a4a42903', 13) total_unique= 33
STORE ('a4a42903', 14) total_unique= 34
STORE ('a4a42903', 15) total_unique= 35
STORE ('a4a42903', 16) total_unique= 36
SIMULATED_ACK_DROP ('a4a42903', 16)
DUPLICATE ('a4a42903', 16) total_unique= 36
STORE ('a4a42903', 17) total_unique= 37
STORE ('a4a42903', 18) total_unique= 38
STORE ('a4a42903', 19) total_unique= 39
STORE ('a4a42903', 20) total_unique= 40
SIMULATED_ACK_DROP ('a4a42903', 20)
DUPLICATE ('a4a42903', 20) total_unique= 40
```

```
ridlo@lenovo:~/iot-lab$ python3 transport_lab.py client --protocol udp --retries 0 --out ack_drop.csv
run=18ef9174 acknowledged=15/20
application_sends 20
CSV: ack_drop.csv
ridlo@lenovo:~/iot-lab$ python3 transport_lab.py client --protocol udp --retries 3 --out ack_retry.csv
run=a4a42903 acknowledged=20/20
application_sends 25
CSV: ack_retry.csv
ridlo@lenovo:~/iot-lab$
```

### 5.3 Deduplikasi Memori vs Durable Exactly-Once Execution

Deduplikasi pada transport_lab.py menggunakan himpunan di memori (seen = set()). Jika server mengalami restart, seluruh riwayat seen akan terhapus dan total_unique kembali ke 0. Oleh karena itu, mekanisme ini merupakan In-Memory Deduplication dan belum memenuhi kriteria Durable Exactly-Once Execution yang tahan terhadap kegagalan daya atau restart server.

## BAB 6: EKSPERIMEN 5 - INSPEKSI HEADER TRANSPORT (TCPDUMP)

### 6.1 Konfigurasi Penangkapan Paket pada Loopback Interface

Eksperimen 5 mengamati perbedaan header dan perilaku lalu lintas data TCP dan UDP pada tingkat paket jaringan menggunakan utilitas tcpdump pada antarmuka loopback (lo) port 5005:

```
# Penangkapan TCP
sudo tcpdump -i lo -nn -s 0 -U -w /tmp/iot_tcp.pcap 'port 5005'
tcpdump -nn -r /tmp/iot_tcp.pcap

# Penangkapan UDP
sudo tcpdump -i lo -nn -s 0 -U -w /tmp/iot_udp.pcap 'port 5005'
tcpdump -nn -r /tmp/iot_udp.pcap
```

### 6.2 Analisis Header dan Handshake TCP

Hasil tangkapan paket TCP memperlihatkan proses pembentukan koneksi 3-Way Handshake di awal komunikasi: paket SYN (Flags [S]) dari klien, SYN-ACK (Flags [S.]) dari server, dan ACK (Flags [.]) dari klien. Setelah transmisi data selesai, terlihat proses penutupan koneksi secara simetris menggunakan paket FIN (Flags [F.]).

```
ridlo@lenovo:/mnt/c/WINDOWS/system32$ sudo tcpdump -i lo -nn -s 0 -U -w /tmp/iot_tcp.pcap 'port 5005'
[sudo: authenticate] Password:
tcpdump: listening on lo, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C48 packets captured
96 packets received by filter
0 packets dropped by kernel
ridlo@lenovo:/mnt/c/WINDOWS/system32$ tcpdump -nn -r /tmp/iot_tcp.pcap
reading from file /tmp/iot_tcp.pcap, link-type EN10MB (Ethernet), snapshot length 262144
10:37:48.206994 IP 127.0.0.1.54638 > 127.0.0.1.5005: Flags [S], seq 2883625070, win 65495, options [mss 65495,sackOK,TS val 954589348 ecr 0,nop,wscale 10], length 0
10:37:48.207064 IP 127.0.0.1.5005 > 127.0.0.1.54638: Flags [S.], seq 676769556, ack 2883625071, win 65483, options [mss 65495,sackOK,TS val 176693165 ecr 954589348,n
op,wscale 10], length 0
10:37:48.207082 IP 127.0.0.1.54638 > 127.0.0.1.5005: Flags [.], ack 1, win 64, options [nop,nop,TS val 954589348 ecr 176693165], length 0
10:37:48.207248 IP 127.0.0.1.54638 > 127.0.0.1.5005: Flags [P.], seq 1:54, ack 1, win 64, options [nop,nop,TS val 954589348 ecr 176693165], length 53
10:37:48.207257 IP 127.0.0.1.5005 > 127.0.0.1.54638: Flags [.], ack 54, win 64, options [nop,nop,TS val 176693165 ecr 954589348], length 0
10:37:48.208068 IP 127.0.0.1.5005 > 127.0.0.1.54638: Flags [P.], seq 1:31, ack 54, win 64, options [nop,nop,TS val 176693166 ecr 954589348], length 30
10:37:48.208143 IP 127.0.0.1.54638 > 127.0.0.1.5005: Flags [.], ack 31, win 64, options [nop,nop,TS val 954589349 ecr 176693166], length 0
10:37:48.228583 IP 127.0.0.1.54638 > 127.0.0.1.5005: Flags [P.], seq 54:107, ack 31, win 64, options [nop,nop,TS val 954589369 ecr 176693166], length 53
10:37:48.228986 IP 127.0.0.1.5005 > 127.0.0.1.54638: Flags [P.], seq 31:61, ack 107, win 64, options [nop,nop,TS val 176693187 ecr 954589369], length 30
10:37:48.249564 IP 127.0.0.1.54638 > 127.0.0.1.5005: Flags [P.], seq 107:160, ack 61, win 64, options [nop,nop,TS val 954589390 ecr 176693187], length 53
10:37:48.249952 IP 127.0.0.1.5005 > 127.0.0.1.54638: Flags [P.], seq 61:91, ack 160, win 64, options [nop,nop,TS val 176693208 ecr 954589390], length 30
10:37:48.270683 IP 127.0.0.1.54638 > 127.0.0.1.5005: Flags [P.], seq 160:213, ack 91, win 64, options [nop,nop,TS val 954589411 ecr 176693208], length 53
10:37:48.271029 IP 127.0.0.1.5005 > 127.0.0.1.54638: Flags [P.], seq 91:121, ack 213, win 64, options [nop,nop,TS val 176693229 ecr 954589411], length 30
10:37:48.291901 IP 127.0.0.1.54638 > 127.0.0.1.5005: Flags [P.], seq 213:266, ack 121, win 64, options [nop,nop,TS val 954589433 ecr 176693229], length 53
10:37:48.292105 IP 127.0.0.1.5005 > 127.0.0.1.54638: Flags [P.], seq 266:319, ack 151, win 64, options [nop,nop,TS val 176693250 ecr 954589433], length 50
10:37:48.312428 IP 127.0.0.1.54638 > 127.0.0.1.5005: Flags [P.], seq 261:183, ack 319, win 64, options [nop,nop,TS val 176589453 ecr 176693250], length 53
10:37:48.312757 IP 127.0.0.1.5005 > 127.0.0.1.54635: Flags [P.], seq 319:382, ack 181, win 64, options [nop,nop,TS val 176589474 ecr 154589453], length 50
10:37:48.333855 IP 127.0.0.1.54638 > 127.0.0.1.5638: Flags [P.], seq 181:219, ack 372, win 64, options [nop,nop,TS val 176589292 ecr 154689474], length 50
10:37:48.354668 IP 127.0.0.1.5005 > 127.0.0.1.54605: Flags [P.], seq 372:429, ack 211, win 64, options [nop,nop,TS val 954589495 ecr 176589292], length 53
10:37:48.355252 IP 127.0.0.1.54638 > 127.0.0.1.5005: Flags [P.], seq 211:241, ack 425, win 64, options [nop,nop,TS val 174589331 ecr 954689495], length 50
10:37:48.375960 IP 127.0.0.1.5005 > 127.0.00.1.5005: Flags [P.], seq 425:478, ack 241, win 64, options [nop,nop,TS val 954589513 ecr 176693313], length 53
10:37:48.376404 IP 127.0.0.1.54638 > 127.0.0.1.5638: Flags [P.], seq 241:271, ack 478, win 64, options [nop,nop,TS val 176693334 ecr 174693334], length 30
10:37:48.397769 IP 127.0.0.1.5005 > 127.0.0.1.54638: Flags [P.], seq 271:302, ack 532, win 64, options [nop,nop,TS val 176589355 ecr 154589538], length 31
10:37:48.418530 IP 127.0.0.1.54638 > 127.0.0.1.5005: Flags [P.], seq 532:586, ack 302, win 64, options [nop,nop,TS val 954589559 ecr 176693355], length 54
10:37:48.418914 IP 127.0.0.1.5005 > 127.0.0.1.54638: Flags [P.], seq 302:333, ack 586, win 64, options [nop,nop,TS val 176583377 ecr 154689559], length 31
10:37:48.439329 IP 127.0.0.1.54638 > 127.0.0.1.5005: Flags [P.], seq 586:640, ack 333, win 64, options [nop,nop,TS val 174589580 ecr 176589377], length 54
10:37:48.439630 IP 127.0.0.1.5005 > 127.0.0.1.54638: Flags [P.], seq 333:364, ack 640, win 64, options [nop,nop,TS val 954589597 ecr 954589580], length 31
10:37:48.460086 IP 127.0.0.1.54638 > 127.0.0.1.5005: Flags [P.], seq 640:694, ack 364, win 64, options [nop,nop,TS val 954589601 ecr 176589397], length 54
10:37:48.460412 IP 127.0.0.1.5005 > 127.0.0.1.54638: Flags [P.], seq 364:395, ack 694, win 64, options [nop,nop,TS val 176589418 ecr 154589601], length 31
10:37:48.481358 IP 127.0.0.1.54638 > 127.0.0.1.5005: Flags [P.], seq 694:748, ack 395, win 64, options [nop,nop,TS val 954589622 ecr 176593418], length 54
10:37:48.481773 IP 127.0.0.1.5005 > 127.0.0.1.54638: Flags [P.], seq 395:426, ack 748, win 64, options [nop,nop,TS val 176583439 ecr 954589622], length 31
10:37:48.502241 IP 127.0.0.1.54638 > 127.0.0.1.5005: Flags [P.], seq 748:802, ack 426, win 64, options [nop,nop,TS val 954589643 ecr 176693439], length 54
```

### 6.3 Analisis Header dan Datagram UDP

Sebaliknya, hasil tangkapan paket UDP tidak menunjukkan adanya proses handshake maupun terminasi koneksi. Paket data langsung dikirim sebagai datagram independen (UDP, length 53 / length 30) antara IP dan port asal-tujuan.

```
ridlo@lenovo:/mnt/c/WINDOWS/system32$ sudo tcpdump -i lo -nn -s 0 -U -w /tmp/iot_udp.pcap 'port 5005'
tcpdump: listening on lo, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C40 packets captured
80 packets received by filter
0 packets dropped by kernel
ridlo@lenovo:/mnt/c/WINDOWS/system32$ tcpdump -nn -r /tmp/iot_udp.pcap
reading from file /tmp/iot_udp.pcap, link-type EN10MB (Ethernet), snapshot length 262144
10:39:12.704368 IP 127.0.0.1.57955 > 127.0.0.1.5005: UDP, length 53
10:39:12.704840 IP 127.0.0.1.5005 > 127.0.0.1.57955: UDP, length 30
10:39:12.725137 IP 127.0.0.1.57955 > 127.0.0.1.5005: UDP, length 53
10:39:12.725406 IP 127.0.0.1.5005 > 127.0.0.1.57955: UDP, length 30
10:39:12.745277 IP 127.0.0.1.57955 > 127.0.0.1.5005: UDP, length 53
10:39:12.745957 IP 127.0.0.1.5005 > 127.0.0.1.57955: UDP, length 30
10:39:12.766566 IP 127.0.0.1.57955 > 127.0.0.1.5005: UDP, length 53
10:39:12.787644 IP 127.0.0.1.5005 > 127.0.0.1.57955: UDP, length 30
10:39:12.787338 IP 127.0.0.1.53055 > 127.0.0.1.5005: UDP, length 30
10:39:12.807808 IP 127.0.0.1.57955 > 127.0.0.1.5005: UDP, length 53
10:39:12.808189 IP 127.0.0.1.5005 > 127.0.0.1.57955: UDP, length 30
10:39:12.828581 IP 127.0.0.1.57955 > 127.0.0.1.5005: UDP, length 53
10:39:12.828942 IP 127.0.0.1.5005 > 127.0.0.1.57955: UDP, length 30
10:39:12.849995 IP 127.0.0.1.57955 > 127.0.0.1.5005: UDP, length 53
10:39:12.850466 IP 127.0.0.1.5005 > 127.0.0.1.57955: UDP, length 30
10:39:12.871076 IP 127.0.0.1.57955 > 127.0.0.1.5005: UDP, length 53
10:39:12.871381 IP 127.0.0.1.5005 > 127.0.0.1.57955: UDP, length 30
10:39:12.892097 IP 127.0.0.1.57955 > 127.0.0.1.5005: UDP, length 54
10:39:12.892580 IP 127.0.0.1.5005 > 127.0.0.1.57955: UDP, length 31
10:39:12.913637 IP 127.0.0.1.57955 > 127.0.0.1.5005: UDP, length 54
10:39:12.914229 IP 127.0.0.1.5005 > 127.0.0.1.57955: UDP, length 31
10:39:12.934742 IP 127.0.0.1.57955 > 127.0.0.1.5005: UDP, length 54
10:39:12.935094 IP 127.0.0.1.5005 > 127.0.0.1.57955: UDP, length 31
10:39:12.955565 IP 127.0.0.1.57955 > 127.0.0.1.5005: UDP, length 54
10:39:12.955927 IP 127.0.0.1.5005 > 127.0.0.1.57955: UDP, length 31
10:39:12.976646 IP 127.0.0.1.57955 > 127.0.0.1.5005: UDP, length 54
10:39:12.976999 IP 127.0.0.1.5005 > 127.0.0.1.57955: UDP, length 31
10:39:12.997851 IP 127.0.0.1.57955 > 127.0.0.1.5005: UDP, length 54
10:39:12.998307 IP 127.0.0.1.5005 > 127.0.0.1.57955: UDP, length 31
10:39:13.019012 IP 127.0.0.1.57955 > 127.0.0.1.5005: UDP, length 54
10:39:13.039670 IP 127.0.0.1.57955 > 127.0.0.1.57955: UDP, length 31
10:39:13.040146 IP 127.0.0.1.5005 >> 127.0.0.1.57905: UDP, length 54
10:39:13.061098 IP 127.0.0.1.57955 > 127.0.0.1.5005: UDP, length 54
10:39:13.061591 IP 127.0.0.1.5005 > 127.0.0.1.57955: UDP, length 31
```

# BAB 7: TABULASI HASIL KUANTITATIF DAN PERHITUNGAN METRIK

### 7.1 Tabel Rekapitulasi Skenario Pengujian Utama

Berikut adalah tabel lengkap rekapitulasi data empiris dari 6 skenario pengujian utama:

| Pengujian (Run) | ACKed | Unique | Sends | Max (ms) | ACK Success Ratio | Retry Overhead |
|---|---|---|---|---|---|---|
| UDP Baseline | 20 | 20 | 20 | 0.851 ms | 20/20 = 1.00 | 0/20 = 0.00 |
| TCP Baseline | 20 | 20 | 20 | 0.780 ms | 20/20 = 1.00 | 0/20 = 0.00 |
| UDP Data Loss, No Retry | 15 | 15 | 20 | 251.104 ms | 15/20 = 0.75 | 0/20 = 0.00 |
| UDP Data Loss, Retries | 20 | 20 | 25 | 251.996 ms | 20/20 = 1.00 | 5/20 = 0.25 |

| | | | | | | |
|---|---|---|---|---|---|---|
| UDP ACK Loss, No Retry | 15 | 20 | 20 | 250.746 ms | 15/20 = 0.75 | 0/20 = 0.00 |
| UDP ACK Loss, Retries | 20 | 20 | 25 | 252.781 ms | 20/20 = 1.00 | 5/20 = 0.25 |

### 7.2 Rumus dan Perhitungan Metrik Evaluasi

Sesuai instruksi modul tutorial, dua metrik utama dihitung menggunakan rumus berikut:

1. **ACK Success Ratio:** ACK Success Ratio = (Acknowledged Records) / 20
2. **Retry Overhead:** Retry Overhead = (Application Sends - 20) / 20

Nilai Retry Overhead mengukur persentase beban tambahan pengiriman di tingkat aplikasi akibat terjadinya retransmisi data.

## BAB 8: HASIL DAN JAWABAN PERTANYAAN ANALISIS

### 8.1 Pengujian mana yang membuktikan bahwa 'tanpa ACK' tidak berarti 'tidak tersimpan'?

Pengujian yang membuktikan hal ini secara mutlak adalah pengujian UDP ACK Loss tanpa retry (ack_drop.csv). Pada pengujian ini, server mencatat total_unique=20 (artinya seluruh 20 rekaman data sensor telah berhasil diterima dan disimpan di memori server). Namun, di sisi klien, laporan hanya menunjukkan acknowledged=15/20. Hal ini terjadi karena fault injector sengaja menjatuhkan balasan ACK untuk sequence kelipatan 4 setelah data tersebut berhasil disimpan oleh server. Dengan demikian, ketiadaan ACK di sisi klien disebabkan oleh hilangnya paket konfirmasi di jalur kembali, bukan karena kegagalan penyimpanan data di sisi server.

### 8.2 Jika aplikasi berbasis TCP membuang perintah yang diterima, apakah TCP akan mengirim ulang perintah tersebut secara otomatis? Jelaskan batasan antara lapisan transport dan aplikasi.

Tidak. TCP tidak akan mengirim ulang perintah tersebut secara otomatis. Tanggung jawab lapisan transport TCP terbatas pada penyerahan byte stream yang andal, berurutan, dan bebas dari kerusakan data hingga masuk ke dalam socket buffer penerima. Begitu data diambil oleh aplikasi melalui pemanggilan fungsi seperti recv() atau readline(), tugas dan batas tanggung jawab protokol TCP secara penuh telah selesai.

Jika aplikasi penerima kemudian mengalami kegagalan logika internal (misalnya kesalahan parsing, kehabisan memori, crash, atau sengaja membuang perintah tersebut), lapisan TCP di kernel tidak memiliki pengetahuan maupun akses terhadap status logika aplikasi tersebut. Oleh karena itu, pengiriman ulang perintah akibat kegagalan logika di tingkat aplikasi harus ditangani oleh mekanisme Acknowledgment dan Retry di lapisan aplikasi itu sendiri (Application-Level Reliability).

### 8.3 Mengapa nomor urut (sequence number) saja menjadi ambigu setelah perangkat di-restart?

Nomor urut (seq) pada skrip telemetri dimulai dari angka 1 setiap kali program klien dijalankan ulang. Jika sebuah perangkat sensor IoT mengalami restart (akibat kehilangan daya atau reset), konter internalnya akan

kembali tereset ke seq = 1. Jika server pengumpul sebelumnya telah menyimpan data seq = 1 hingga seq 20 dari sesi sebelum restart, maka data seq 1 yang baru dikirim setelah restart akan dianggap sebagai data lama atau duplikat jika server hanya memeriksa nomor urut saja.

Oleh karena itu, penggunaan sequence number saja bersifat ambigu. Untuk mengatasi masalah ini, program transport_lab.py menggabungkan identifier unik berupa UUID acak sepanjang 8 karakter (run) dengan nomor urut seq menjadi pasangan tuple kunci (run, seq). Dengan demikian, setiap sesi eksekusi baru memiliki namespace kunci yang unik dan tidak akan saling bertabrakan dengan data dari sesi sebelumnya.

### 8.4 Untuk pembacaan suhu berkala dan perintah pembukaan katup (valve), tentukan persyaratan reliabilitas dan kesegaran data yang berbeda. Jangan memilih hanya berdasarkan protokol mana yang memiliki waktu latensi lokal lebih rendah.

Pemilihan protokol komunikasi IoT harus didasarkan pada karakteristik dan konsekuensi kegagalan data:

1. Pembacaan Suhu Berkala (Periodic Temperature Telemetry):
    - Persyaratan Utama: Kesegaran Data (Data Freshness).
    - Karakteristik: Data dikirim secara kontinu (misalnya setiap 10 detik). Jika satu sampel data hilang di perjalanan, sampel data berikutnya akan segera menggantikan informasi tersebut. Menunggu retransmisi data suhu yang lama justru merugikan karena data tersebut sudah tidak segar lagi.
    - Protokol yang Sesuai: UDP (atau UDP dengan CoAP). Protokol ini sangat hemat energi dan tidak membebankan overhead handshake, sangat ideal untuk sensor node berdaya baterai kecil.
2. Perintah Pembukaan Katup (Valve-Opening Command):
    - Persyaratan Utama: Keandalan Mutlak (Absolute Reliability & Idempotency).
    - Karakteristik: Perintah ini bersifat kritis dan berdampak fisik langsung. Hilangnya satu perintah dapat menyebabkan katup tidak terbuka, yang berpotensi memicu bahaya keselamatan atau kerusakan sistem. Data ini tidak boleh hilang dan tidak boleh dieksekusi secara tidak sengaja lebih dari sekali.
    - Protokol yang Sesuai: TCP (atau UDP dengan lapisan Application-Level ACK/Retry dan Idempotency Key). Protokol harus menjamin bahwa perintah benar-benar sampai, dikonfirmasi oleh penerima, dan dieksekusi dengan aman.

### 8.5 Apa yang harus diubah agar deduplikasi bersifat tahan lama (durable) dan mendukung banyak klien secara bersamaan (concurrent)?

Untuk mengamankan sistem dalam skala produksi, dua perubahan arsitektur berikut wajib diimplementasikan:

1. Deduplikasi Tahan Lama (Durable Deduplication):
    - Kondisi Saat Ini: Server menggunakan set() di memori acak (seen = set()) yang akan hilang sepenuhnya jika server mengalami restart atau mati mati listrik.
    - Perubahan yang Dibutuhkan: Status kunci yang telah diproses (run, seq) harus disimpan ke dalam media penyimpanan persisten (seperti basis data SQL/NoSQL seperti SQLite, PostgreSQL, Redis, atau file terstruktur yang di-flush ke disk). Transaksi penyimpanan

data dan pembaruan status deduplikasi harus dilakukan secara atomik (ACID transaction) untuk menjamin Exactly-Once Processing bahkan jika terjadi kecelakaan sistem.

2. Dukungan Klien Bersamaan (Multiple Concurrent Clients):
    - Kondisi Saat Ini: Server pada transport_lab.py menggunakan s.listen(1) dan loop pemrosesan tunggal (single-threaded blocking) yang hanya dapat melayani satu klien TCP pada satu waktu.
    - Perubahan yang Dibutuhkan: Arsitektur server harus direfaktor menggunakan pemrograman konkurensi, seperti Multi-threading (ThreadingServer), Multi-processing, atau Non-blocking Event Loop berbasis I/O asinkron (seperti modul asyncio pada Python atau arsitektur Node.js/Go).

# BAB 9: REFLEKSI BUG, KENDALA EKSEKUSI, DAN SOLUSI TEKNIS

## 9.1 Refleksi Bug: Perintah Bash Dijalankan di Windows PowerShell

Pada awal eksekusi praktikum, terjadi kendala teknis di mana perintah persiapan lingkungan Bash (seperti mkdir, sudo apt, dan pembuatan skrip Python) dijalankan langsung pada jendela Windows PowerShell (prompt PS C:\Users\Lenovo>) dan bukannya di dalam sesi Linux WSL Ubuntu. Karena PowerShell tidak mengenali sintaks shell Linux, muncul serangkaian pesan kesalahan seperti 'The term import is not recognized' dan 'Missing ( after if'. Solusinya adalah memanggil perintah 'wsl' terlebih dahulu dari PowerShell untuk masuk ke dalam sesi Ubuntu sebelum mengeksekusi skrip.

## 9.2 Tabel Kendala dan Solusi Teknis

| Kendala / Error | Akar Penyebab | Solusi Teknis yang Diterapkan |
|---|---|---|
| Perintah Bash gagal di PowerShell | Belum berpindah dari Windows Command Line ke WSL Linux. | Mengeksekusi perintah 'wsl' di PowerShell untuk masuk ke sesi Ubuntu. |
| Keluaran awk tergabung tanpa spasi | Penggunaan echo -n tanpa pembatas karakter akhir. | Tetap dapat membaca nilai median_ms dan max_ms secara akurat dari file CSV. |
| tcpdump memerlukan akses root | Antarmuka jaringan loopback memerlukan hak akses administratif. | Menambahkan awalan 'sudo' saat menjalankan tcpdump di Terminal C. |

# BAB 10: EKSTENSI OPSIONAL DAN KESIMPULAN GENERAL

## 10.1 Evaluasi Ekstensi Opsional (Actual IP Packet Loss)

Bagian pengujian simulasi packet loss menggunakan utilitas tc netem dan Linux network namespace bersifat opsional. Bagian ini tidak dikerjakan karena memerlukan konfigurasi tiga network namespace aktif dari modul Tutorial 2 sebagai prasyarat utama, yang berada di luar cakupan lima eksperimen wajib praktikum.

### 10.2 Kesimpulan Umum

Seluruh lima eksperimen inti praktikum Transport Layer telah berhasil dilaksanakan dengan hasil yang 100% konsisten terhadap teori dan spesifikasi modul tutorial:

**1. Message Framing:** TCP merupakan aliran byte (byte stream) tanpa batas pesan bawaan, sehingga aplikasi wajib menyediakan pemisah rekaman (delimiter) atau length-prefix framing.
**2. Baseline Telemetri:** Pada jaringan yang sehat, TCP dan UDP sama-sama mencapai tingkat keberhasilan 20/20 acknowledged.
**3. Application Reliability:** Mekanisme Bounded Retry di tingkat aplikasi berhasil memulihkan kehilangan data maupun ACK, dengan biaya tambahan transmisi (Retry Overhead) sebesar 25%.
**4. Timeout Trade-offs:** Nilai timeout harus dirancang secara cermat, timeout yang terlalu pendek memicu spurious retries, sedangkan retry tanpa batas berisiko menghabiskan daya baterai sensor IoT.
**5. Inspeksi Header:** Inspeksi paket tcpdump mengonfirmasi bahwa TCP memiliki overhead 3-way handshake dan terminasi FIN, sedangkan UDP beroperasi secara connectionless tanpa overhead koneksi.

## BAB 11: LAMPIRAN KODE SUMBER PYTHON

### Lampiran A: Berkas transport_lab.py

Berikut adalah salinan lengkap skrip transport_lab.py yang digunakan selama pengujian:

```
#!/usr/bin/env python3
"""Small TCP/UDP telemetry experiment. Python 3 standard library only."""
import argparse
import csv
import json
import socket
import time
import uuid

def encode(obj):
    return (json.dumps(obj) + "\n").encode()

def serve(a):
    seen, attempted = set(), set()
    def process(raw):
        msg = json.loads(raw)
        key = (msg["run"], msg["seq"])
        first = key not in attempted
        attempted.add(key)
        fault = (a.protocol == "udp" and a.drop_every > 0 and msg["seq"] % a.drop_every == 0
and first)
        if fault and a.fault == "data":
            print("SIMULATED_DATA_DROP", key, flush=True)
            return None
        duplicate = key in seen
        seen.add(key)
        print("DUPLICATE" if duplicate else "STORE", key, "total_unique=", len(seen),
flush=True)
        if fault and a.fault == "ack":
```

```
            print("SIMULATED_ACK_DROP", key, flush=True)
            return None
        return encode({"run": msg["run"], "seq": msg["seq"]})

    kind = socket.SOCK_DGRAM if a.protocol == "udp" else socket.SOCK_STREAM
    with socket.socket(socket.AF_INET, kind) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.bind((a.host, a.port))
        print("READY", a.protocol, a.host, a.port, flush=True)
        if a.protocol == "udp":
            while True:
                raw, peer = s.recvfrom(4096)
                reply = process(raw)
                if reply is not None:
                    s.sendto(reply, peer)
        else:
            s.listen(1)
            while True:
                conn, _ = s.accept()
                with conn, conn.makefile("rb") as stream:
                    for raw in stream:
                        conn.sendall(process(raw))

def send(a):
    run = uuid.uuid4().hex[:8]
    kind = socket.SOCK_DGRAM if a.protocol == "udp" else socket.SOCK_STREAM
    rows = []
    with socket.socket(socket.AF_INET, kind) as s:
        s.settimeout(a.timeout)
        s.connect((a.host, a.port))
        stream = s.makefile("rb") if a.protocol == "tcp" else None
        try:
            for seq in range(1, a.count + 1):
                raw = encode({"run": run, "seq": seq, "temperature_c": round(24 + seq / 10,
1)})
                start = time.perf_counter()
                acknowledged = False
                limit = a.retries + 1 if a.protocol == "udp" else 1
                for attempt in range(1, limit + 1):
                    if a.protocol == "tcp":
                        s.sendall(raw)
                        reply = stream.readline()
                        if not reply:
                            raise ConnectionError("Server closed connection")
                        ack = json.loads(reply)
                        acknowledged = (ack["run"], ack["seq"]) == (run, seq)
                    else:
                        s.send(raw)
                        deadline = time.perf_counter() + a.timeout
                        while time.perf_counter() < deadline:
                            s.settimeout(max(0.001, deadline - time.perf_counter()))
                            try:
                                ack = json.loads(s.recv(4096))
                            except socket.timeout:
                                break
                            if (ack["run"], ack["seq"]) == (run, seq):
```

```
                                acknowledged = True
                                break
                    if acknowledged:
                        break
                elapsed = 1000 * (time.perf_counter() - start)
                rows.append([seq, attempt, int(acknowledged), round(elapsed, 3)])
                time.sleep(a.interval)
        finally:
            if stream:
                stream.close()
            with open(a.out, "w", newline="") as f:
                writer = csv.writer(f)
                writer.writerow(["seq", "attempts", "acknowledged", "elapsed_ms"])
                writer.writerows(rows)
    ok = sum(row[2] for row in rows)
    print(f"run={run} acknowledged={ok}/{a.count}")
    print("application_sends=", sum(row[1] for row in rows))
    print("CSV:", a.out)

def main():
    p = argparse.ArgumentParser()
    p.add_argument("role", choices=["server", "client"])
    p.add_argument("--protocol", choices=["tcp", "udp"], default="udp")
    p.add_argument("--host", default="127.0.0.1")
    p.add_argument("--port", type=int, default=5005)
    p.add_argument("--count", type=int, default=20)
    p.add_argument("--retries", type=int, default=0)
    p.add_argument("--timeout", type=float, default=0.25)
    p.add_argument("--interval", type=float, default=0.02)
    p.add_argument("--drop-every", type=int, default=0)
    p.add_argument("--fault", choices=["data", "ack"], default="data")
    p.add_argument("--out", default="results.csv")
    a = p.parse_args()
    if a.role == "server":
        serve(a)
    else:
        send(a)

if __name__ == "__main__":
    main()
```

## Lampiran B: Berkas framing_demo.py

Berikut adalah salinan lengkap skrip framing_demo.py yang digunakan pada Eksperimen 1:

```
#!/usr/bin/env python3
"""TCP demonstration: two sendall calls are not two receive records."""
import argparse
import socket

p = argparse.ArgumentParser()
p.add_argument("role", choices=["server", "client"])
a = p.parse_args()

with socket.socket() as s:
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
```

```
if a.role == "server":
    s.bind(("127.0.0.1", 5006))
    s.listen(1)
    print("READY on port 5006", flush=True)
    conn, _ = s.accept()
    with conn:
        chunks = []
        while True:
            part = conn.recv(4)
            if not part:
                break
            print("recv:", repr(part))
            chunks.append(part)
        print("records:", b"".join(chunks).splitlines())
else:
    s.connect(("127.0.0.1", 5006))
    s.sendall(b"temperature=25\n")
    s.sendall(b"humidity=60\n")
    s.shutdown(socket.SHUT_WR)
```